

Punto 3.1 — Fundamentos de LINQ to Entities

Contexto del proyecto

Este checkpoint continúa `M02/PROYECTO/2.12`. Se mantiene la separación Domain/Application/Infrastructure/Console y el esquema se obtiene con migraciones, no con `EnsureCreated()`.

Objetivo práctico

Implementar y ejecutar **Fundamentos de LINQ to Entities** sobre `AceriaData`, inspeccionando resultados reales y, cuando corresponde, el SQL generado por EF Core 8.

Paso 1: Abrir el checkpoint

```
cd M03/PROYECTO/3.1
dotnet restore AceriaData.sln
```

La solución contiene los cuatro proyectos y puede abrirse directamente en Visual Studio 2022/2026 compatible con .NET 8.

Paso 2: Comprobar la historia de migraciones

```
dotnet ef migrations list --project src/AceriaData.Infrastructure --startup-project src/
AceriaData.Console
```

El listado debe conservar las migraciones de M1 y M2, incluida `M2_2_12_Architecture`. LINQ y las estrategias de carga de M3 no cambian el esquema, por lo que M3 no inventa migraciones vacías.

Paso 3: Revisar los cambios de este punto

Los elementos trazados en este estado son: `Consulta`, `ObtenerSqlFundamentos`. El contrato crece de forma acumulativa, salvo en 3.12, donde el seam docente `IQueryable` se retira del puerto público y la composición queda encapsulada en `Infrastructure`.

Paso 4: Implementar y estudiar el caso de uso

Archivo: `src/AceriaData.Application/ConsultasLinqUseCase.cs`

```

using AceriaData.Application.Interfaces;

namespace AceriaData.Application.UseCases;

public sealed class ConsultasLinqUseCase
{
    private readonly IUnidadDeTrabajo _unidad;
    public ConsultasLinqUseCase(IUnidadDeTrabajo unidad) => _unidad = unidad;

    public void Ejecutar()
    {
        Console.WriteLine("=== FUNDAMENTOS DE LINQ TO ENTITIES ===");
        var enMemoria = _unidad.Ordenes.ObtenerTodas().Where(o => o.Cliente == "Constructora del
Norte").ToList();
        var consulta = _unidad.Ordenes.Consulta().Where(o => o.Cliente == "Constructora del
Norte").OrderBy(o => o.FechaCreacion);
        Console.WriteLine("Consulta IQueryable construida: aún no se ha materializado.");
        var enSql = consulta.ToList();
        if (enMemoria.Count != 3 || enSql.Count != 3) throw new InvalidOperationException("Comparación
IEnumerable/IQueryable inesperada.");
        Console.WriteLine($"Enumerable: {enMemoria.Count} | IQueryable: {enSql.Count}");
        Console.WriteLine(_unidad.Ordenes.ObtenerSqlFundamentos());
    }
}

```

Explicación línea a línea del caso de uso 3.1

Línea 1: `using AceriaData.Application.Interfaces;` → Importa los puertos de Application, en especial IUnidadDeTrabajo e IOrdenRepositorio, que desacoplan el caso de uso de EF Core.

Línea 3: `namespace AceriaData.Application.UseCases;` → Declara el espacio de nombres de la capa a la que pertenece el archivo.

Línea 5: `public sealed class ConsultasLinqUseCase` → Declara el caso de uso concreto que coordina la demostración del punto.

Línea 6: `{` → Abre el bloque de la clase, método, inicializador u opción declarada inmediatamente antes; su cierre delimita exactamente ese ámbito.

Línea 7: `private readonly IUnidadDeTrabajo _unidad;` → Guarda el puerto de unidad de trabajo que da acceso al repositorio sin depender de DbContext.

Línea 8: `public ConsultasLinqUseCase(IUnidadDeTrabajo unidad) => _unidad = unidad;` → Recibe la unidad de trabajo mediante inyección de dependencias.

Línea 10: `public void Ejecutar()` → Define la operación docente que ejecutará el composition root.

Línea 11: `{` → Abre el bloque de la clase, método, inicializador u opción declarada inmediatamente antes; su cierre delimita exactamente ese ámbito.

Línea 12: `Console.WriteLine("=== FUNDAMENTOS DE LINQ TO ENTITIES ===");` → Escribe una evidencia legible en consola para poder inspeccionar el flujo o el SQL.

Línea 13: `var enMemoria = _unidad.Ordenes.ObtenerTodas().Where(o => o.Cliente == "Constructora del Norte").ToList();` → Materializa primero todas las órdenes y aplica después el filtro en memoria, creando deliberadamente el contraste con la consulta IQueryable.

Línea 14: `var consulta = _unidad.Ordenes.Consulta().Where(o => o.Cliente == "Constructora del Norte").OrderBy(o => o.FechaCreacion);` → Compone filtro y ordenación sobre IQueryable sin materializar; EF Core conserva el árbol de expresión hasta la ejecución.

Línea 15: `Console.WriteLine("Consulta IQueryable construida: aún no se ha materializado.");` → Escribe una evidencia legible en consola para poder inspeccionar el flujo o el SQL.

Línea 16: `var enSql = consulta.ToList();` → Materializa la secuencia en una lista; si el origen sigue siendo IQueryable, aquí se ejecuta SQL.

Línea 17: `if (enMemoria.Count != 3 || enSql.Count != 3) throw new`

`InvalidOperationException("Comparación IEnumerable/IQueryable inesperada.");` → Comprueba una condición E2E y hace fallar la ejecución si los resultados no coinciden con los datos esperados.

Línea 18: `Console.WriteLine($"Enumerable: {enMemoria.Count} | IQueryable: {enSql.Count}");` → Escribe una evidencia legible en consola para poder inspeccionar el flujo o el SQL.

Línea 19: `Console.WriteLine(_unidad.Ordenes.ObtenerSqlFundamentos());` → Escribe una evidencia legible en consola para poder inspeccionar el flujo o el SQL.

Línea 20: `}` → Cierra el bloque sintáctico abierto previamente y termina el ámbito correspondiente.

Línea 21: `}` → Cierra el bloque sintáctico abierto previamente y termina el ámbito correspondiente.

Paso 5: Preparar y ejecutar el composition root

Archivo: `src/AceriaData.Console/Program.cs`

```
using AceriaData.Application.UseCases;
using AceriaData.ConsoleApp;
using AceriaData.Infrastructure;
using AceriaData.Infrastructure.Persistence;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;

var configuration = new ConfigurationBuilder()
    .SetBasePath(AppContext.BaseDirectory)
    .AddJsonFile("appsettings.json", optional: false)
    .AddEnvironmentVariables()
    .Build();

var connectionString = configuration.GetConnectionString("AceriaDB")
    ?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB");

var services = new ServiceCollection();
services.AddAceriaInfrastructure(connectionString);
services.AddScoped<ConsultasLinqUseCase>();

using var provider = services.BuildServiceProvider(new ServiceProviderOptions
{
    ValidateOnBuild = true,
    ValidateScopes = true
});
using var scope = provider.CreateScope();

var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
context.Database.EnsureDeleted();
context.Database.Migrate();
DemoData.Seed(context);
context.ChangeTracker.Clear();

var useCase = scope.ServiceProvider.GetRequiredService<ConsultasLinqUseCase>();
useCase.Ejecutar();

Console.WriteLine("3.1 OK");
```

Explicación línea a línea de Program.cs 3.1

Línea 1: `using AceriaData.Application.UseCases;` → Importa los casos de uso de Application para poder registrarlos y resolverlos desde el composition root.

Línea 2: `using AceriaData.ConsoleApp;` → Importa DemoData, responsable de sembrar el conjunto determinista usado por las pruebas E2E del checkpoint.

Línea 3: `using AceriaData.Infrastructure;` → Importa la extensión AddAceriaInfrastructure, que registra DbContext, repositorio y unidad de trabajo.

Línea 4: `using AceriaData.Infrastructure.Persistence;` → Importa `AceriaDbContext` para preparar la base de demostración desde el `composition root`.

Línea 5: `using Microsoft.EntityFrameworkCore;` → Importa las extensiones de EF Core utilizadas por `Migrate`, `ChangeTracker` y las operaciones de consulta del `checkpoint`.

Línea 6: `using Microsoft.Extensions.Configuration;` → Importa `ConfigurationBuilder` y las extensiones necesarias para cargar `appsettings.json` y variables de entorno.

Línea 7: `using Microsoft.Extensions.DependencyInjection;` → Importa `ServiceCollection`, `AddScoped`, `CreateScope` y `GetRequiredService` para componer el grafo de dependencias.

Línea 9: `var configuration = new ConfigurationBuilder()` → Inicia la construcción de la configuración de la aplicación.

Línea 10: `.SetBasePath(AppContext.BaseDirectory)` → Fija como base el directorio de ejecución para localizar `appsettings.json`.

Línea 11: `.AddJsonFile("appsettings.json", optional: false)` → Carga la configuración JSON obligatoria que contiene la cadena de conexión.

Línea 12: `.AddEnvironmentVariables()` → Permite sobrescribir configuración mediante variables de entorno.

Línea 13: `.Build();` → Finaliza la construcción y devuelve la configuración inmutable que se usará para leer `ConnectionStrings:AceriaDB`.

Línea 15: `var connectionString = configuration.GetConnectionString("AceriaDB")` → Obtiene la cadena `AceriaDB` desde la configuración.

Línea 16: `?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB");` → Detiene el arranque con un error explícito si no existe la cadena de conexión `AceriaDB`, evitando ejecutar el ejercicio con una configuración incompleta.

Línea 18: `var services = new ServiceCollection();` → Crea el contenedor de servicios del `composition root`.

Línea 19: `services.AddAceriaInfrastructure(connectionString);` → Registra `DbContext`, repositorio y unidad de trabajo de `Infrastructure`.

Línea 20: `services.AddScoped<ConsultasLinqUseCase>();` → Registra el caso de uso con ciclo de vida `Scoped` para compartir el mismo ámbito que `DbContext`.

Línea 22: `using var provider = services.BuildServiceProvider(new ServiceProviderOptions` → Construye el proveedor de servicios y lo declara con `using` para liberar correctamente los servicios `IDisposable` al terminar el proceso.

Línea 23: `{` → Abre el bloque de la clase, método, inicializador u opción declarada inmediatamente antes; su cierre delimita exactamente ese ámbito.

Línea 24: `ValidateOnBuild = true,` → Obliga al contenedor a comprobar durante la construcción que los servicios registrados pueden resolverse.

Línea 25: `ValidateScopes = true` → Activa la validación de ciclos de vida para detectar, por ejemplo, un servicio `Scoped` consumido incorrectamente desde un `Singleton`.

Línea 26: `});` → Cierra las opciones y completa la llamada que construye el proveedor de servicios.

Línea 27: `using var scope = provider.CreateScope();` → Crea el ámbito `Scoped` que compartirá `AceriaDbContext`, `IUnidadDeTrabajo` y el caso de uso durante esta ejecución.

Línea 29: `var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();` → Resuelve el `DbContext` para preparar la base de demostración.

Línea 30: `context.Database.EnsureDeleted();` → Elimina solo la base de demostración para que el E2E comience desde un estado reproducible; no crea el esquema.

Línea 31: `context.Database.Migrate();` → Aplica la historia real de migraciones heredada de M2 para crear o actualizar el esquema.

Línea 32: `DemoData.Seed(context);` → Inserta el conjunto determinista de órdenes, planchas, detalles y aleaciones usado por los E2E.

Línea 33: `context.ChangeTracker.Clear();` → Desacopla las entidades sembradas para que las consultas se materialicen de nuevo desde SQL.

Línea 35: `var useCase = scope.ServiceProvider.GetRequiredService<ConsultasLinqUseCase>();` → Resuelve el caso de uso concreto registrado para este checkpoint.

Línea 36: `useCase.Ejecutar();` → Ejecuta la demostración y sus aserciones E2E.

Línea 38: `Console.WriteLine("3.1 OK");` → Escribe una evidencia legible en consola para poder inspeccionar el flujo o el SQL.

Paso 6: Compilar el estado acumulativo

```
dotnet build AceriaData.sln --configuration Release
```

La compilación debe terminar sin errores. Application no referencia Entity Framework Core; todas las llamadas específicas de EF permanecen en Infrastructure o en el composition root.

Paso 7: Ejecutar el E2E contra SQL Server LocalDB

```
dotnet run --project src/AceriaData.Console/AceriaData.Console.csproj --configuration Release
```

El proceso elimina la base de demostración, aplica `Database.Migrate()`, inserta un dataset determinista, limpia el Change Tracker y ejecuta el caso de uso. El marcador final obligatorio es:

```
3.1 OK
```

Paso 8: Analizar resultado y SQL

El caso de uso contiene aserciones que hacen fallar el proceso si cambian los resultados esperados. Cuando el punto estudia traducción SQL, el repositorio devuelve `ToQueryString()` y la consola imprime exactamente la sentencia que EF Core 8 ha construido para SQL Server. No se usa una sentencia SQL ficticia como prueba.

Paso 9: Diagnosticar un error común

Error: Materializar antes de filtrar mueve el trabajo a memoria y oculta el SQL que se quería estudiar.

Para diagnosticarlo, compara el método del repositorio, la salida E2E y el SQL obtenido con `ToQueryString()` cuando esté disponible. No cambies simultáneamente datos, query y expectativa: modifica una sola variable para poder atribuir la causa.

Laboratorio adicional del punto 3.1

Diagnóstico técnico

El objetivo no es sólo obtener el mismo número de órdenes con `IEnumerable<T>` e `IQueryable<T>`, sino comprobar dónde se ejecuta el filtro. Si se materializa primero, el SQL trae todas las filas visibles y el predicado se aplica en memoria; si se mantiene `IQueryable<T>`, el predicado llega al `WHERE` de SQL Server.

Reto resuelto y verificación adicional

Construye una consulta de órdenes pendientes del Norte, ordenadas de más reciente a más antigua y proyectadas a número y cliente. El pipeline correcto es `Where → OrderByDescending → Select → ToList`. Antes de materializar, usa `ToQueryString()` y verifica `WHERE`, `ORDER BY` y una proyección mínima.

Errores comunes revisados

Error	Causa	Solución
Filtro se aplica en memoria	Se llamó a <code>ToList</code> antes del filtro	Aplicar <code>ToList</code> al final
Expresión no traducida en filtro u operación de servidor	Se usó lógica que el proveedor no puede traducir fuera de la proyección superior	Reescribir con expresiones traducibles o cambiar explícitamente a evaluación cliente cuando esté justificado
Múltiples consultas	Se materializó varias veces	Materializar una sola vez al final
<code>ToQueryString</code> ejecuta la consulta	No, solo la construye	Verificar que no se llama a <code>ToList</code>
<code>IQueryable</code> expuesto en el repositorio	Anti-patrón	Encapsular en métodos específicos (Módulo 4)
Falta el using de EF Core	El método <code>ToQueryString</code> no está disponible	Añadir <code>using Microsoft.EntityFrameworkCore;</code>

Analogía operativa

LINQ to Entities en una acería es como el lenguaje que usa el jefe de planta para pedir informes al archivo central. El jefe no va al archivo a buscar los documentos: envía una orden con las condiciones (cliente, estado, fecha) y el archivo devuelve solo los documentos que cumplen esas condiciones. Si el jefe pidiera todos los documentos y luego los filtrara él mismo, perdería tiempo y esfuerzo. Eso es lo que ocurre en este ejemplo cuando primero se llama a `ToList()` y después se filtra como `IEnumerable`: los datos ya están materializados y el filtro posterior se ejecuta en el cliente. `IEnumerable` por sí solo no significa que toda secuencia esté previamente cargada en memoria. La ejecución diferida es como preparar la orden de búsqueda antes de enviarla: el jefe puede añadir condiciones hasta que esté seguro, y solo entonces la envía. La materialización es el momento en que se envía la orden y se recibe la respuesta. `ToQueryString()` es como pedir una copia de la orden sin enviarla: permite revisar el SQL que EF Core ha construido sin ejecutar esa consulta.

Paso 10: Reto resuelto y conexión con el siguiente punto

Reto: Añade un segundo filtro opcional por estado sin materializar hasta el final y compara el SQL.

Cómo activar el reto en el checkpoint

Estos bloques ya existen comentados en el checkpoint. Actívalos sólo para reproducir esta variante y restaura después el estado original.

Archivo: `src/AceriaData.Application/Interfaces.cs`

Marcador: RETO M03 3.1 - PUERTO SQL OPCIONAL

```
// Se activa coordinadamente con el repositorio y el caso de uso del reto.
string ObtenerSqlRetoFundamentos(string cliente, string? estado);
```

Archivo: `src/AceriaData.Infrastructure/Repositories/Repositories.cs`

Marcador: RETO M03 3.1 - SQL OPCIONAL Y PROYECCION MINIMA

```
// Mantiene IQueryable hasta el final, aplica estado solo cuando existe
// y permite comparar el SQL sin ejecutar la consulta.
public string ObtenerSqlRetoFundamentos(string cliente, string? estado)
{
    var consulta = _context.OrdenesFabricacion
        .Where(o => o.Cliente == cliente);

    if (!string.IsNullOrEmpty(estado))
    {
        consulta = consulta.Where(o => o.Estado == estado);
    }

    return consulta
        .OrderByDescending(o => o.FechaCreacion)
        .Select(o => new { o.NumeroOrden, o.Cliente })
        .ToString();
}
```

Archivo: src/AceriaData.Application/ConsultasLinqUseCase.cs

Marcador: RETO M03 3.1 - FILTRO OPCIONAL SIN MATERIALIZAR

```
// Activa este bloque junto con los bloques RETO 3.1 del puerto y del repositorio.
// Demuestra el Paso 10 y el laboratorio adicional: segundo filtro opcional,
// materialización al final, orden descendente y proyección mínima.
string? estadoOpcional = "Pendiente";

var consultaReto = _unidad.Ordenes.Consulta()
    .Where(o => o.Cliente == "Constructora del Norte");

if (!string.IsNullOrEmpty(estadoOpcional))
{
    consultaReto = consultaReto.Where(o => o.Estado == estadoOpcional);
}

var resultadoReto = consultaReto
    .OrderByDescending(o => o.FechaCreacion)
    .Select(o => new { o.NumeroOrden, o.Cliente })
    .ToList();

if (resultadoReto.Count != 2)
    throw new InvalidOperationException("Reto 3.1: se esperaban dos órdenes pendientes del
Norte.");

var sqlReto = _unidad.Ordenes.ObtenerSqlRetoFundamentos("Constructora del Norte",
estadoOpcional);
Console.WriteLine($"Reto 3.1 OK | Filas: {resultadoReto.Count}");
Console.WriteLine(sqlReto);
```

Comprobación esperada: filtro opcional, materialización final, orden descendente, proyección mínima y SQL real.

La solución está representada por el estado acumulativo del propio checkpoint y sus métodos de repositorio. Tras validar 3.1 OK, el siguiente estado parte exactamente de esta solución y añade

Consultas básicas: Where, OrderBy y ThenBy.